

HALTING PROBLEM

3)ii. State the halting problem of TMs.

Prove that the halting problem of Turing Machine over $\{0,1\}^*$ as unsolvable.[BL2]

[CO6] [7]

ANSWER:

Definition

The **Halting Problem** asks whether a given **Turing Machine (TM)** will:

1. **Halt (stop)** on a given input, or
2. **Run forever** without stopping.

Formal statement

For a Turing Machine M and input w :

$HALT = \{(M, w) | M \text{ halts on input } w\}$

- The Halting Problem is **undecidable (unsolvable)**.

Simple example

Suppose we have:

$M = \text{A Turing Machine}$

$w = 101$

We want to know:

Will M stop when input is 101?



YES → HALT

NO → RUN FOREVER

- There is **no general algorithm** that can correctly answer this for **every** Turing Machine and every input.

TO PROVE THAT THE HALTING PROBLEM OF TURING MACHINE OVER $\{0,1\}^*$ AS UNSOLVABLE:

Step 1: Assume a machine H exists

Suppose there is a Turing Machine **H** that can solve the Halting Problem.

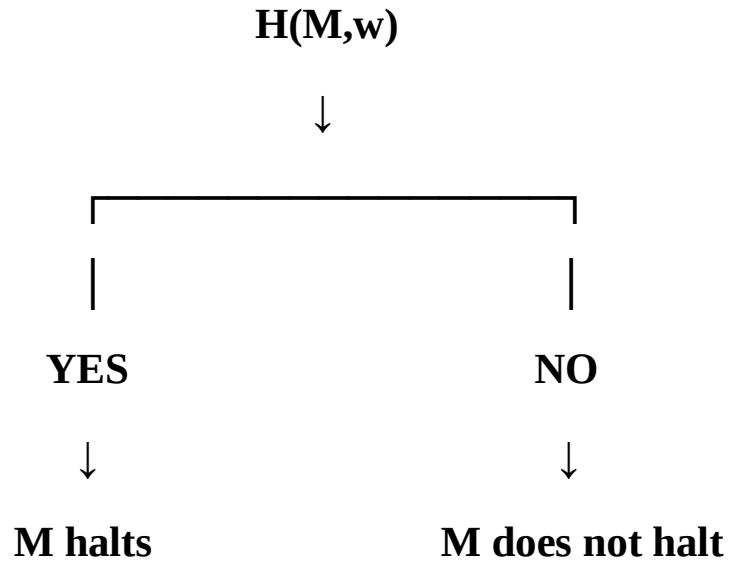
It takes:

$H(M,w)$

where:

- **M = Turing Machine**
- **w = binary input such as 101, 001, etc.**

H gives:



Step 2: Create another machine D

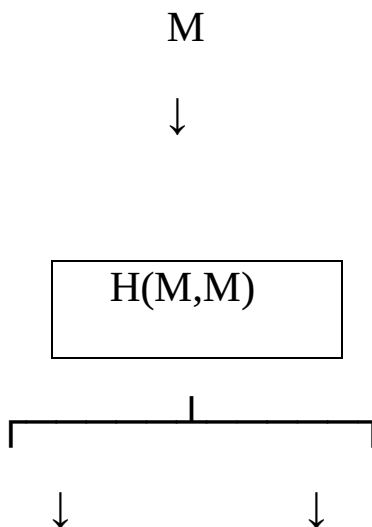
Now construct a new machine **D**.

D takes a Turing Machine **M** as input.

It asks H:

$H(M, M)$

Then D does the **opposite** of H's answer.



YES NO



Loop

Halt

forever

So:

- If H says **YES** → D **loops forever**
- If H says **NO** → D **halts**

Step 3: Give D to itself:

Now give D as its own input:

D(D)D(D)

There are two possibilities.

Case 1: H says YES

H(D,D)=YESH(D,D)=YES

H says:

"D will halt."

But D is designed to do the opposite.

So, D **loops forever**.

CONTRADICTION.

Case 2: H says NO

$H(D,D)=NO$ $H(D,D)=NO$

H says:

"D will not halt."

But D is designed to do the opposite.

So ,D **halts**.

CONTRADICTION AGAIN.

Step 4: Conclusion:

- > Both possible answers of H produce a contradiction.
- > Therefore, our assumption that **H exists is wrong**.

Hence:

Halting Problem is undecidable/unsolvable

For $\{0,1\}^*$

$\rightarrow \{0,1\}^*$ means **all possible finite strings made using 0 and 1.**

Examples:

ϵ
0
1
00
01
10
11
000

5. Describe about Halting Problem in detail with suitable example. [BL2] [CO6] [13]

Answer:

1. Introduction:

The **Halting Problem** is a famous problem in **Theory of Computation**.

It asks:

Can we determine whether a given Turing Machine will stop or run forever for a particular input?

The answer is **No**.

Therefore, the Halting Problem is called an **undecidable problem**.

HALTING:

Halting means the Turing Machine **stops its execution**.

For example:

Input $\rightarrow$ Turing Machine $\rightarrow$ Processing $\rightarrow$ STOP

If the machine never stops:

Input $\rightarrow$ Turing Machine $\rightarrow$ Processing $\rightarrow$ Processing $\rightarrow$ Processing $\rightarrow$...

It runs forever.

Statement of Halting Problem

Let:

- M = Turing Machine

- w = input string

The Halting Problem asks whether:

M halts on input w

For a Turing Machine M and input w :

$HALT = \{(M, w) | M \text{ halts on input } w\}$

EXAMPLE PROBLEM :

to prove that the halting problem of turing machine over $\{0,1\}^*$ as unsolvable:

Step 1: Assume a machine H exists

Suppose there is a Turing Machine **H** that can solve the Halting Problem.

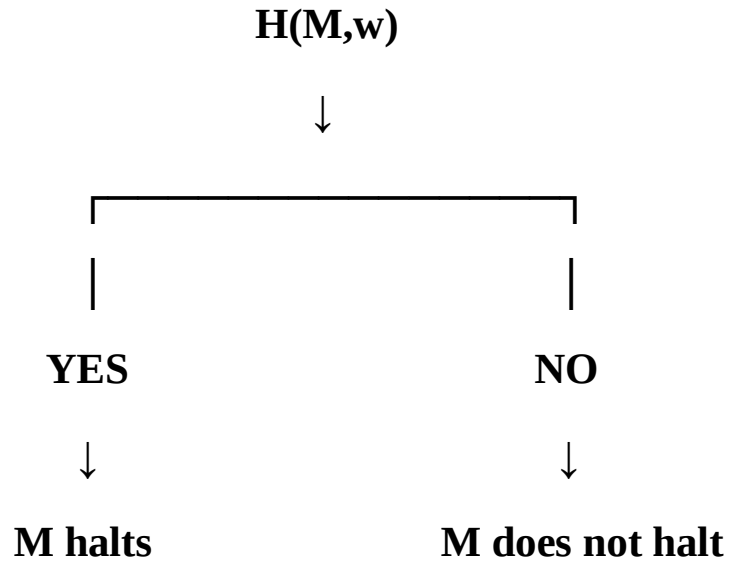
It takes:

$H(M, w)$

where:

- **M = Turing Machine**
- **w = binary input such as 101, 001, etc.**

H gives:



Step 2: Create another machine D

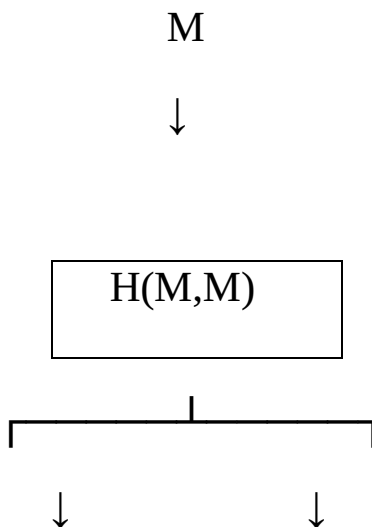
Now construct a new machine **D**.

D takes a Turing Machine **M** as input.

It asks H:

$H(M, M)$

Then D does the **opposite** of H's answer.



YES NO



Loop

Halt

forever

So:

- If H says **YES** → D **loops forever**
- If H says **NO** → D **halts**

Step 3: Give D to itself:

Now give D as its own input:

D(D)D(D)

There are two possibilities.

Case 1: H says YES

H(D,D)=YESH(D,D)=YES

H says:

"D will halt."

But D is designed to do the opposite.

So, D **loops forever**.

CONTRADICTION.

Case 2: H says NO

$H(D,D)=NO$

H says:

"D will not halt."

But D is designed to do the opposite.

So, D **halts**.

CONTRADICTION AGAIN.

Step 4: Conclusion:

- > Both possible answers of H produce a contradiction.
- > Therefore, our assumption that **H exists is wrong**.

Hence:

Halting Problem is undecidable/unsolvable

For $\{0,1\}^*$

$\rightarrow \{0,1\}^*$ means **all possible finite strings made using 0 and 1.**

Examples:

ϵ
0
1
00
01
10
11
000

HALTING PROBLEM – UNDECIDABILITY PROOF

15-Mark Answer | Theory of Computation

1. Introduction

The Halting Problem is a famous problem in Theory of Computation. It asks whether we can create a program that can always determine whether another program will stop (halt) or run forever. Alan Turing proved that no such general program can exist. Therefore, the Halting Problem is undecidable.

2. What is the Halting Problem?

Suppose we have a Turing Machine M and an input w . We want to determine whether M will halt (stop its execution) or run forever (never stop).

The Halting Problem can be represented as:

$$\text{HALT} = \{(M, w) \mid M \text{ halts on input } w\}$$

The question is: Is there a Turing Machine that can always decide whether M halts on w ? Turing proved that the answer is NO.

3. Example

Consider the following simple programs.

Program A:

Start

Print "Hello"

Stop

Program A clearly halts.

Program B:

Start

while(true)

Print "Hello"

Program B never halts.

The Halting Problem asks whether a general machine can correctly identify both cases for every possible program and input.

4. Assumption

To prove that the Halting Problem is undecidable, we use proof by contradiction.

Assume that a machine called H exists.

H is called the Halting Machine. It takes two inputs: a program or Turing Machine M and an input w. It is written as $H(M, w)$.

H gives the following answers:

YES – M will halt on input w.

NO – M will run forever on input w.

We assume that H can always correctly decide the Halting Problem.

5. Construct a New Machine D

Now we create another machine called D. D uses the machine H.

The working of D is:

D(M):
Run H(M, M)

If H says YES:
Loop forever

If H says NO:
Halt

Notice that D gives its own description as input. That is, we consider D(D).
This creates a contradiction.

6. Case 1: Suppose H says YES

Suppose $H(D, D) = \text{YES}$.

This means H predicts that D will halt when D is given itself as input.

But according to the definition of D, if H says YES, D loops forever.
Therefore, D does not halt.

H predicted that D would halt, but D actually does not halt. **This is a contradiction.**

7. Case 2: Suppose H says NO

Now suppose $H(D, D) = \text{NO}$.

This means H predicts that D will run forever when given itself as input.

But according to the definition of D, if H says NO, D halts.

Therefore, D halts. But H predicted that D would never halt. Again, this is a contradiction.

8. Contradiction Table

H's prediction	What D does	Result
YES – D will halt	D loops forever	Contradiction
NO – D will loop	D halts	Contradiction

forever		
---------	--	--

9. Conclusion

Our original assumption was that a machine H exists that can always decide whether any program will halt. But we showed that this assumption leads to a contradiction.

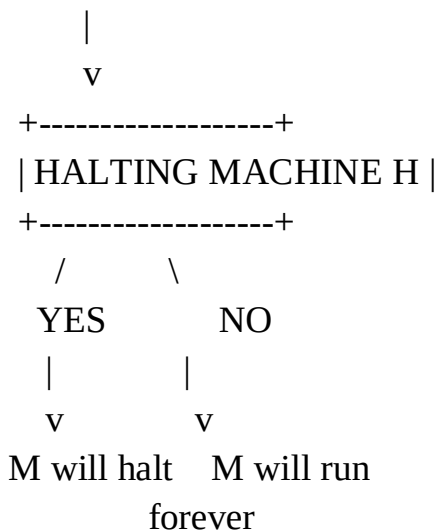
No such machine H can exist.

Therefore, the Halting Problem is Undecidable.

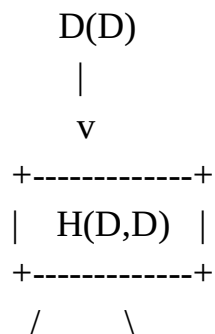
10. Simple Diagram

General Halting Problem:

Program M + Input w



Contradiction using D:



YES	NO
v	v
D loops	D halts
forever	
^	
+-----+	
CONTRADICTION	

11. Importance of the Halting Problem

- Some computational problems cannot be solved by any algorithm.
- A computer cannot always determine whether an arbitrary program will stop.
- It is a fundamental example of an undecidable problem.
- It helps us understand the limits of computers and algorithms.

HALTING PROBLEM IS UNDECIDABLE